

Bloc 4

Maintenir l'application logicielle en condition opérationnelle

Dossier de certification — projet CountryTravel

Candidat	Lucas Lafourcade
École	Ynov Campus
Projet support	CountryTravel — plateforme de découverte de destinations de voyage
Code source	github.com/aref400/countryTravel
Nature du dossier	Document autonome — preuves reproduites dans le corps du dossier
Date de remise	Août 2026

Sommaire

Sommaire	2
Table de correspondance avec le référentiel	3
1. Contexte d'exploitation	3
2. Processus de mise à jour des dépendances	4
Fréquence	4
Périmètre logiciel concerné	4
Type de mise à jour : manuel et contrôlé	4
Règles d'intégration sécurisée	4
3. Système de supervision	4
3.1 Périmètre et sondes	5
3.2 Indicateurs de suivi	5
3.3 Seuils d'alerte	6
3.4 Modalité de signalement	6
4. Traitement des anomalies en production	7
4.1 Processus de collecte et de consignation	7
4.2 Fiche de consignation d'une anomalie	8
4.3 Traitement de l'anomalie via l'intégration et le déploiement continu	9
Cheminement dans la chaîne CI/CD	9
5. Maintenance du logiciel	9
5.1 Recommandations argumentées d'amélioration	9
5.2 Journal des versions	10
5.3 Problème résolu en collaboration avec le support client	10
Contribution des différentes parties prenantes	10
6. Synthèse	11
Annexe — Localisation des éléments dans le dépôt	11

Table de correspondance avec le référentiel

Ce dossier couvre l'intégralité des éléments attendus au titre du Bloc 4. Chaque exigence du référentiel d'évaluation est mise en regard de la compétence visée, du caractère éliminatoire ou non de celle-ci, et de la section qui la traite. Les preuves (extraits de code, réponses des sondes, fiches, journal) sont reproduites directement dans le corps du dossier.

Élément attendu du livrable	Compétence	Éliminatoire	Section
Processus de mise à jour des dépendances	C4.1.1	—	2
Système de supervision	C4.1.2	Oui	3
Processus de collecte et de consignation des anomalies	C4.2.1	Oui	4.1
Fiche de consignation d'une anomalie	C4.2.1	Oui	4.2
Traitement d'une anomalie détectée	C4.2.2	—	4.3
Recommandations argumentées d'amélioration	C4.3.1	—	5.1
Journal de version	C4.3.2	Oui	5.2
Problème résolu avec le support client	C4.3.3	—	5.3

Dossier autonome

Conformément au format attendu (dossier de 20 pages maximum), les procédures, extraits de code, réponses des sondes et exemplaire du journal nécessaires à l'évaluation sont reproduits directement dans les sections qui suivent. Le code source complet et la documentation détaillée résident dans le dépôt Git du projet (localisation rappelée en annexe).

1. Contexte d'exploitation

CountryTravel est une application web d'aide au choix d'une destination de voyage : moteur de recommandation par critères, catalogue de pays, avis et carte mondiale. Ce dossier présente la gestion du monitoring, du traitement des anomalies et de la maintenance de l'application, développée et déployée en production au cours de la formation.

L'application est déployée en continu sur trois composants, cibles de la maintenance en condition opérationnelle :

Composant	Technologie	Hébergement
API REST	NestJS 11 + Prisma	Railway

Composant	Technologie	Hébergement
Base de données	PostgreSQL	Railway
Front (SPA)	React 19 + Vite	Netlify

Toute livraison passe par une pull request vers `main`, validée automatiquement par la CI (lint, tests, build des deux applications), puis déployée automatiquement à la fusion. Ce socle d'intégration et de déploiement continu, décrit au [4.3](#), est le pivot de la maintenance : correctifs et évolutions empruntent le même chemin outillé.

2. Processus de mise à jour des dépendances

Compétence C4.1.1. La maintenance des dépendances suit un processus périodique et contrôlé, décrit ci-dessous selon les trois axes attendus : fréquence, périmètre et type de mise à jour.

Fréquence

Revue mensuelle des dépendances, et **immédiate** en cas d'alerte de sécurité (npm audit).

Périmètre logiciel concerné

Les deux espaces de travail du monorepo : `apps/api` (NestJS, Prisma, Passport...) et `apps/front` (React, Vite, librairie de cartographie...), ainsi que les actions GitHub des workflows CI/CD.

Type de mise à jour : manuel et contrôlé

Le choix est assumé de ne pas automatiser les montées de version, afin d'en maîtriser les impacts :

```
npm audit      # état des vulnérabilités connues
npm audit fix  # corrections sans changement de version majeure
npm outdated   # dépendances en retard (informationnel)
```

Règles d'intégration sécurisée

- **Jamais de npm audit fix --force sans analyse préalable** (exploitabilité réelle, chemin de dépendance) : une montée majeure automatique peut casser l'application.
- Après toute mise à jour : `npx tsc --noEmit` puis suite de tests complète avant commit.
- **Les montées de version majeures** (NestJS, Prisma, React, Vite) se font isolément, une à la fois, dans une PR dédiée, où la CI valide l'absence de régression.

Exemple réellement traité

Une vulnérabilité critique remontée par npm audit sur une dépendance transitive a été corrigée de façon ciblée (version 0.4.0 du journal des versions, cf. [5.2](#)), sans montée de version majeure, après vérification de la suite de tests.

3. Système de supervision

Compétence C4.1.2 — éliminatoire. Le dispositif de supervision décrit ci-dessous vise à garantir la disponibilité permanente du logiciel en détectant au plus tôt toute indisponibilité ou dégradation.

3.1 Périmètre et sondes

Quatre sondes couvrent les trois composants :

Sonde	Nature	Finalité
GET /api/v1/health/live	Endpoint applicatif	Le process API répond-il ? (liveness)
GET /api/v1/health	Endpoint applicatif (Terminus)	L'API peut-elle servir ? Teste la base (SELECT 1) et mesure sa latence (readiness)
Sentry	SDK API + front	Capture des erreurs applicatives non gérées
Moniteur d'uptime	Service externe	Disponibilité perçue de l'extérieur, avec alerte

Les endpoints de santé sont implémentés côté API avec la librairie `@nestjs/terminus` (module `apps/api/src/health/`). Chaque indicateur Terminus constitue une sonde : ici la connexion PostgreSQL, dont la sonde renvoie le statut et le temps de réponse. Cœur de la sonde de readiness :

```
async isHealthy(key: string) {
  const indicator = this.healthIndicatorService.check(key);
  try {
    const start = Date.now();
    await this.prisma.$queryRaw`SELECT 1`; // requête triviale, sans effet de bord
    return indicator.up({ responseTime: Date.now() - start });
  } catch (error) {
    return indicator.down({ message: (error as Error).message });
  }
}
```

Réponse de la readiness lorsque la base est disponible (**HTTP 200**) :

```
{
  "status": "ok",
  "info": { "database": { "status": "up", "responseTime": 3 } },
  "details": { "database": { "status": "up", "responseTime": 3 } }
}
```

Lorsque la base est injoignable, la sonde renvoie **HTTP 503** avec `status: "down"` : le moniteur externe le détecte et déclenche l'alerte.

```
{
  "status": "error",
  "error": {
    "database": { "status": "down",
      "message": "Invalid `this.prisma.$queryRaw()` invocation" }
  }
}
```

3.2 Indicateurs de suivi

Disponibilité (uptime %), temps de réponse de la readiness (ms), statut de la base (up/down), nombre et taux d'erreurs runtime (Sentry), uptime du process (s). Ces indicateurs sont adaptés à la typologie du logiciel : une application web à trafic modéré dont la disponibilité dépend de la base.

Ils constituent les **critères de qualité et de performance** suivis en exploitation : la disponibilité et le temps de réponse prolongent en production les critères de performance vérifiés lors du développement, et le taux d'erreurs mesure la fiabilité perçue par l'utilisateur. Les seuils associés sont fixés au [3.3](#).

3.3 Seuils d'alerte

Condition	Niveau	Action
/health ≠ 200 sur 2 vérifications consécutives	Critique	Alerte immédiate (indisponibilité)
Temps de réponse readiness > 2000 ms	Alerte	Notification
Temps de réponse readiness > 500 ms (sur la durée)	Vigilance	Surveillance (dégradation)
Nouvelle erreur remontée par Sentry	Alerte	Notification e-mail
Disponibilité mensuelle < 99 %	Alerte	Analyse a posteriori

Avec un intervalle de vérification de 5 minutes, la règle « 2 vérifications consécutives » déclenche l'alerte en environ 10 minutes : elle évite les faux positifs d'un incident réseau transitoire tout en garantissant une détection rapide.

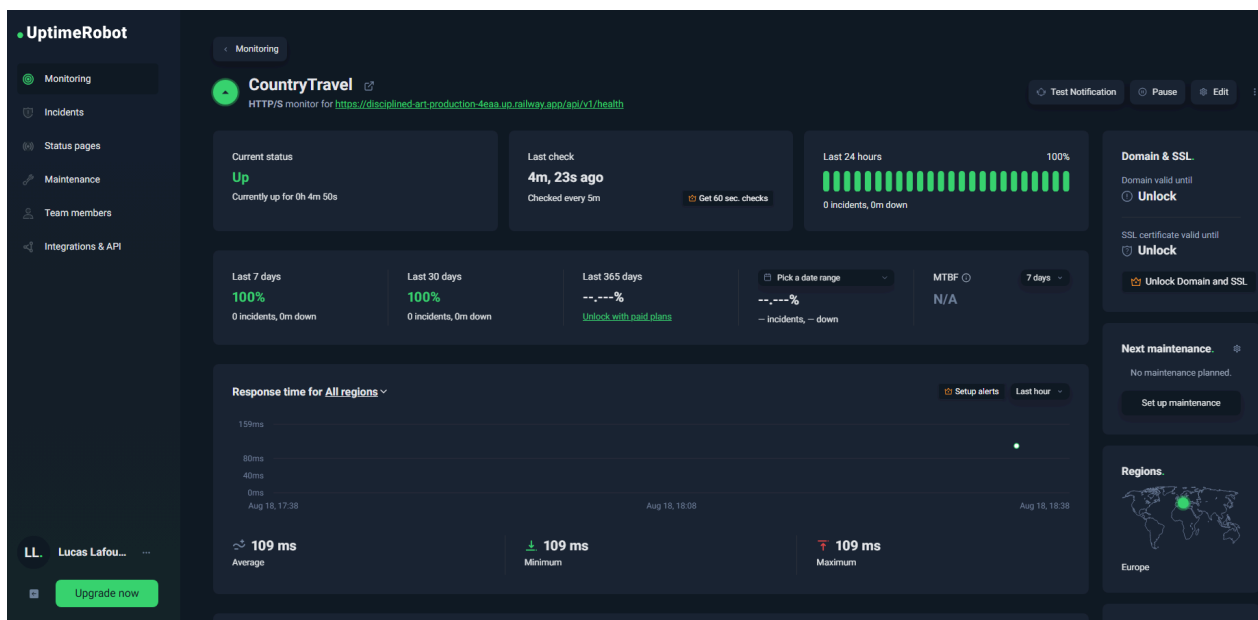


Figure 1 — Moniteur Uptime Robot

3.4 Modalité de signalement

Le moniteur d'uptime notifie par **email** à l'ouverture et à la résolution d'un incident ; Sentry notifie par e-mail sur nouvelle erreur, avec la stack trace et le contexte de diagnostic. L'activation de Sentry est conditionnée à une variable d'environnement (SENTRY_DSN côté API, VITE_SENTRY_DSN côté front) : sans elle, le SDK est un no-op, sans impact sur l'application.

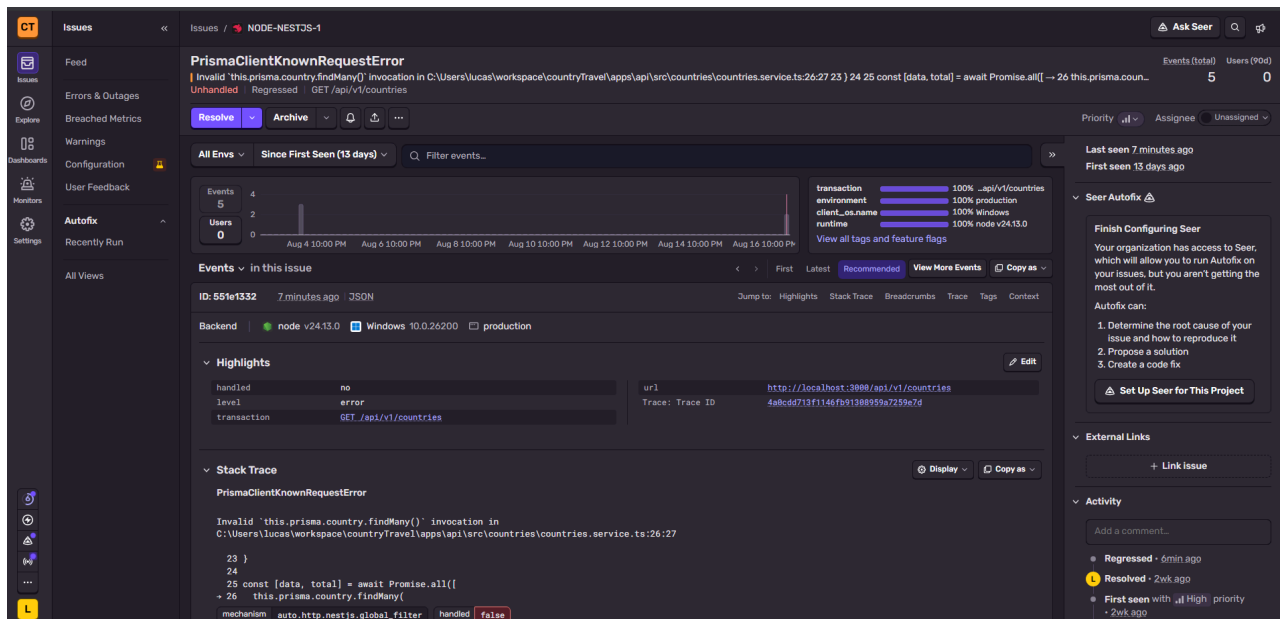


Figure 2 — Erreur capturée dans Sentry avec la stack trace

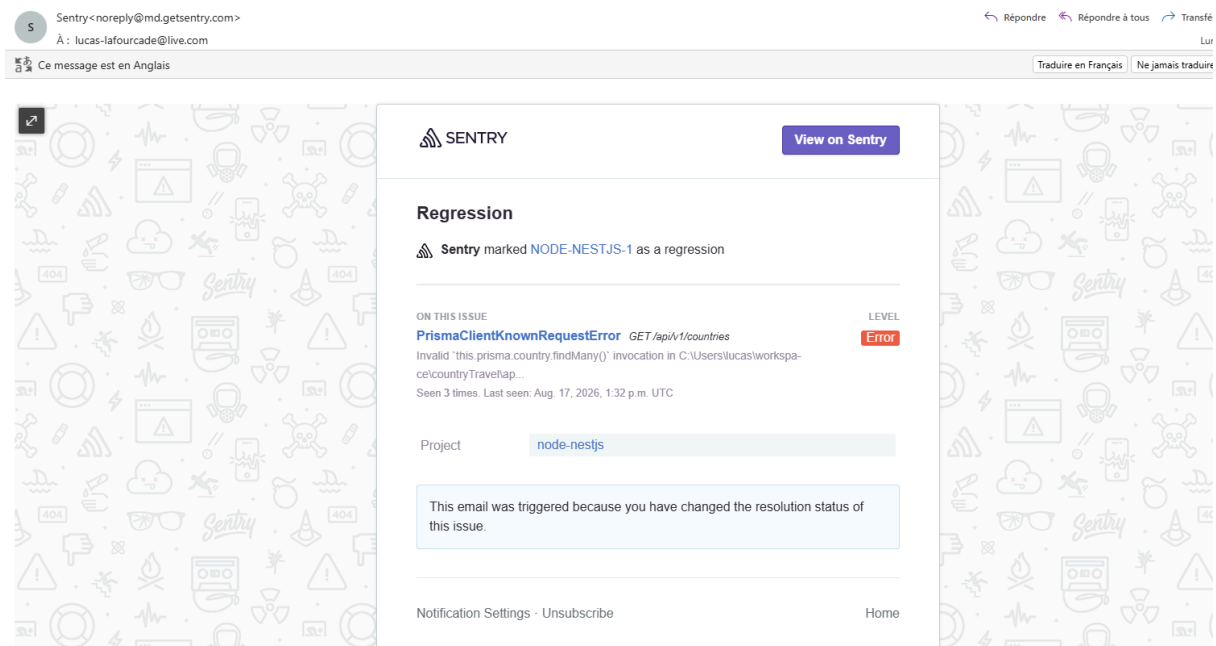


Figure 3 — Notification e-mail émise par Sentry

4. Traitement des anomalies en production

4.1 Processus de collecte et de consignation

Compétence C4.2.1 — éliminatoire. Le processus de traitement des anomalies, structuré et adapté à la typologie du logiciel, comporte cinq étapes :

- 1. Détection** — scénario du cahier de recettes en échec, retour utilisateur, signalement de la supervision (Sentry, moniteur d'uptime) ou échec de la CI.
- 2. Consignation** — ouverture d'une issue GitHub via un gabarit de rapport de bogue standardisé, qui impose les informations nécessaires à la reproduction : description, étapes de reproduction,

comportement attendu face à l'observé, gravité, périmètre, environnement, source de détection.
L'anomalie confirmée est reportée en fiche dans un registre consolidé des bogues du projet.

3. **Analyse** — identification de la cause racine dans le code.
4. **Correction** — développement sur une branche dédiée (fix/...).
5. **Vérification** — rejeu du scénario de recette concerné et revue de la CI avant fusion.

L'issue GitHub est l'**outil de collecte** : elle garantit qu'aucune information indispensable à la reproduction n'est omise à la saisie.

Figure 4 — Gabarit d'issue GitHub « Rapport de bogue », outil de collecte des anomalies

4.2 Fiche de consignation d'une anomalie

Compétence C4.2.1 — éliminatoire. Fiche d'une anomalie réellement rencontrée (référéncée BUG-02), consignée au format du gabarit :

Titre	Perte de session après rechargement de la page
Gravité	Majeur
Périmètre	Front (React) — authentification
Source	Scénario de recette AUTH-18
Environnement	Chrome / Windows 11 — application déployée (Netlify)

Étapes de reproduction : se connecter avec des identifiants valides, constater l'accès à l'espace authentifié, rafraîchir la page (F5), observer le retour à l'état non connecté.

Comportement attendu : la session est conservée après rafraîchissement tant que le token est valide.

Comportement observé : déconnexion silencieuse à chaque rechargement.

Analyse (cause racine) : `setAuth` (`apps/front/src/shared/store/auth.store.ts`) mettait à jour l'état Zustand (`user`, `accessToken`) mais n'écrivait jamais le token dans `localStorage` — seul `logout` y touchait. Le token n'était donc jamais présent pour être relu à l'initialisation de l'application.

Préconisation de correction : persister le token dans `localStorage` au moment de `setAuth` (avant la mise à jour du state), afin qu'il soit réhydraté au démarrage de l'application.

Ces éléments — reproduction déterministe, analyse de la cause racine et préconisation — réunissent les informations nécessaires pour corriger l'anomalie. Le correctif effectivement mis en place et déployé est présenté au [4.3](#).

4.3 Traitement de l'anomalie via l'intégration et le déploiement continu

Compétence C4.2.2. On reprend l'anomalie BUG-02 consignée au [4.2](#) (où figurent l'analyse de la cause racine et la préconisation) pour illustrer le traitement de bout en bout, qui tire profit du processus d'intégration et de déploiement continu.

Correctif mis en place. `setAuth` écrit désormais explicitement le token dans `localStorage` avant de mettre à jour le state, ce qui permet sa réhydratation au démarrage et résout la perte de session.

Cheminement dans la chaîne CI/CD

1. Correction développée sur une branche dédiée `fix/...`
2. **Ouverture d'une pull request vers main :** le workflow d'intégration continue (`pr.yml`) exécute automatiquement lint, tests et build des deux applications. La PR n'est fusionnable que si tout est vert — garde-fou contre l'introduction d'une régression par le correctif lui-même.
3. Vérification fonctionnelle : rejeu du scénario AUTH-18 (connexion puis F5, l'utilisateur reste authentifié).
4. Fusion sur `main`, déclenchement automatique du workflow de déploiement continu (`deploy-front.yml`, Netlify).
5. En cas de problème détecté après coup, le retour arrière s'effectue par « Publish deploy » sur le déploiement antérieur (Netlify) ou `git revert`, sans interruption de service.

Le correctif, décrit ci-dessus et tracé dans l'historique Git (commit dédié), **résout l'anomalie** tout en s'appuyant sur le pipeline pour garantir la non-régression.

5. Maintenance du logiciel

5.1 Recommandations argumentées d'amélioration

Compétence C4.3.1. Ces axes découlent des limites connues du projet et des indicateurs de supervision. Chacun est argumenté en termes de gain, de coût de mise en œuvre et de délai — ils sont réalistes et réalisables au regard du projet.

Recommandation	Gain attendu	Coût / délai
Chargement différé de la page /carte	Réduction sensible du poids du bundle initial (> 1,2 Mo), premier affichage plus rapide sur toutes les autres pages	Faible — quelques heures, une PR ciblée
Automatiser les migrations Prisma dans <code>deploy-api.yml</code>	Supprime l'unique étape manuelle post-déploiement (source d'erreur et d'oubli), déploiements plus fiables	Faible — modification d'un workflow existant

Recommandation	Gain attendu	Coût / délai
Seuil de couverture ciblé sur la logique métier du front	Prévention des régressions sur la couche à plus fort risque	Faible — configuration Vitest
Page de statut publique et historique d'uptime	Transparence sur la disponibilité, preuve de suivi, communication en cas d'incident	Très faible — activation d'une option du moniteur
Test de charge (montée en concurrence)	Valider la tenue sous charge réelle, dimensionner l'hébergement	Moyen — mise en place d'un scénario de charge

Priorisation proposée : le chargement différé de /carte (meilleur rapport gain/coût, améliore l'expérience de tous les utilisateurs), puis l'automatisation des migrations (fiabilise l'exploitation).

5.2 Journal des versions

Compétence C4.3.2 — éliminatoire. Les évolutions sont consignées dans un journal des versions (CHANGELOG.md, format Keep a Changelog, versionnage SemVer). Chaque version datée détaille les améliorations apportées : nouvelles fonctionnalités (rubrique Ajouté), correctifs rattachés aux fiches de bogues (Corrigé) et corrections de sécurité (Sécurité). Exemple du journal :

[Non publié] Ajouté : supervision (endpoints /health liveness et readiness, intégration Sentry), gabarit de consignation des anomalies, journal des versions.

[0.4.0] – 2026-07-24 Corrigé : BUG-11 (validation du secret de refresh au démarrage). Sécurité : correction d'une vulnérabilité critique remontée par npm audit. Documentation : dossier Bloc 2, manuels, qualité et sécurité.

[0.3.0] – 2026-07-17 Ajouté : carte mondiale interactive (CT-018), module « pays visités » (CT-019), module « avis » (CT-020), page de profil (CT-021), pipeline de tests en CI. Corrigé : BUG-05 à BUG-10 (dont échec de build CI, page blanche sur URL inconnue, expiration de session silencieuse).

[0.2.0] – 2026-05-14 Ajouté : catalogue et détail des pays (CT-013), moteur de recommandation et tirage aléatoire (CT-014), gestion des recommandations sauvegardées (CT-015 à CT-017). Corrigé : BUG-01 à BUG-04 (crash sur cache invalide, perte de session, retour arrière, 404 sur liens directs).

[0.1.0] – 2026-04-23 Ajouté : socle NestJS, React/Vite et Prisma, intégration et déploiement continus (Railway, Netlify), authentification (CT-05, CT-07), seed des pays et critères (CT-08, CT-09), API pays et moteur de score (CT-010, CT-011), page d'accueil (CT-012).

Le journal documente ainsi les correctifs déployés et retrace, version par version, l'ensemble des évolutions du logiciel.

5.3 Problème résolu en collaboration avec le support client

Compétence C4.3.3. Contexte du retour. Lors d'une campagne de tests utilisateurs, un testeur a signalé une difficulté d'usage sur la sauvegarde d'une recommandation (BUG-09) : après avoir validé l'enregistrement, il ne savait pas si l'action avait réussi — la fenêtre se fermait sans aucun message, impossible de distinguer un succès d'une annulation. Le problème à résoudre était donc un défaut d'ergonomie générant de l'incertitude et des sauvegardes répétées.

Contribution des différentes parties prenantes

- **Le testeur (utilisateur)** a remonté le ressenti et le contexte précis d'usage, permettant de qualifier le problème comme un manque de retour d'action plutôt qu'un dysfonctionnement technique.

- **Le développeur** a analysé le comportement, reproduit le cas et identifié trois défauts secondaires associés (message d'erreur persistant d'un essai précédent, bouton actif pendant la requête, libellé de debug résiduel).
- **La consignation puis la vérification** se sont appuyées sur le cahier de recettes (scénarios SAV-10, SAV-12) comme référentiel partagé entre le rôle de test et le rôle de développement.

Résolution apportée. Remplacement des indicateurs épars par une machine à états explicite (inactive → saving → success | error) pilotant la fenêtre de sauvegarde : message de confirmation en cas de succès (avec lien vers le tableau de bord), message d'erreur avec bouton « Réessayer » en cas d'échec, bouton désactivé pendant la requête, réinitialisation de l'état à chaque ouverture. Cinq tests unitaires couvrant les quatre états ont été ajoutés pour prévenir toute régression.

Résultat. Le retour d'action est désormais sans ambiguïté ; le rejeu des scénarios de recette confirme la résolution. L'échange a par ailleurs mis en évidence, dans la foulée, l'anomalie d'expiration de session silencieuse (BUG-10), traitée à son tour — illustration de la valeur du retour utilisateur dans l'amélioration continue du logiciel.

6. Synthèse

La maintenance en condition opérationnelle de CountryTravel repose sur un dispositif cohérent : une **supervision** multi-sondes (santé applicative, base, erreurs, disponibilité externe) avec seuils et signalement ; un **processus de traitement des anomalies** outillé (issues GitHub, registre consolidé) adossé à la chaîne d'intégration et de déploiement continu ; et une **maintenance évolutive** tracée par un journal de versions et nourrie par les retours utilisateurs. Les axes d'amélioration identifiés, priorisés par rapport gain/coût, en assurent la poursuite.

Annexe — Localisation des éléments dans le dépôt

Le présent dossier est autonome. Pour information, les éléments techniques correspondants résident dans le dépôt Git du projet (non joint au dossier) :

Élément	Emplacement
Sondes de santé	apps/api/src/health/
Supervision détaillée	docs/supervision.md
Processus et registre des anomalies	docs/plan-correction-bogues.md
Gabarit de collecte	.github/ISSUE_TEMPLATE/bug_report.yml
Journal des versions complet	CHANGELOG.md
Mise à jour et exploitation	docs/manuel-mise-a-jour.md
Qualité et performance	docs/qualite-performance.md
Intégration et déploiement continu	.github/workflows/